

企业级应用软件开发与开发

课程项目报告

项目名称：基于 LangChain4j 的多智能体问
卷系统设计与实现

方向：Agentic AI 原生开发

姓名：卢嘉诚

学号：2025303029

专业：软件工程

老师：戚欣

2026 年 6 月 22 日

目 录

一、选题背景与设计思想.....	1
1.1 问题定义.....	1
1.2 现有方案不足.....	1
1.3 项目核心价值.....	1
1.4 技术路线.....	1
二、Specs 规格文档	2
2.1 Product Spec	2
2.2 Architecture Spec.....	2
2.3 API Spec	3
三、系统架构与设计.....	4
3.1 核心架构图.....	4
3.2 Agent 交互流程.....	4
3.3 数据流设计.....	5
四、关键实现与代码展示.....	6
4.1 Agent 核心循环.....	6
4.2 工具定义.....	7
4.3 配置文件.....	8
4.4 AI IDE 使用截图.....	8
五、测试与评估.....	10
5.1 功能测试.....	10
5.2 Agent 行为评估.....	10
5.3 Benchmark 设计	10
5.4 测试用例表.....	11
5.5 Demo 截图说明.....	11
六、系统升级与扩展.....	14
6.1 可扩展架构.....	14
6.2 下一阶段发展计划.....	14
6.3 AI 能力演进路径	15
七、课程总结.....	15
7.1 个人学习收获.....	15
7.2 工程思维转变.....	15
7.3 对课程的建议.....	16

一、选题背景与设计思想

1.1 问题定义

问卷系统是企业、学校和组织收集信息的常见工具，但传统问卷平台通常强调“手动配置字段、人工设计逻辑、后台查看结果”。当用户希望快速创建一份具有业务语义的问卷时，需要理解题型、字段名称、选项、校验规则、发布状态、提交限制等细节；当管理员希望了解“上周提交量最高的问卷”“某类问卷平均得分”“最近登录异常用户”等问题时，又需要在多个页面之间筛选、导出或编写 SQL。项目要解决的核心问题是：如何让非技术用户以自然语言完成问卷创建、数据分析和运维查询，同时保证系统具备企业级的安全性、可扩展性和可维护性。

1.2 现有方案不足

项目早期版本偏向传统动态表单系统，后端业务逻辑较集中，返回结构不够标准化，密码加密、JWT 配置、输入验证、日志体系和异步处理能力需要强化；前端动态表达式执行、分页和数据加载也存在进一步治理空间。市面上的通用问卷工具虽然功能成熟，但通常缺少与内部业务数据、操作日志和权限体系的深度结合；单纯接入大模型的聊天机器人又容易停留在文本问答层面，无法真实执行系统操作。CC Survey 的设计目标不是简单增加一个聊天入口，而是让 Agent 与现有问卷业务模块形成可控、可审计的工具调用关系。

1.3 项目核心价值

第一，降低使用门槛。管理员可以输入“帮我创建一个员工满意度调查，包含工作环境、薪资福利、团队氛围三个维度”，系统自动生成问卷结构并进入预览确认流程。第二，提高分析效率。用户可以直接问“统计一下最近一周各问卷提交量”，DataAnalysisAgent 或 NaturalQueryAgent 将查询真实数据库并返回结果。第三，增强可治理性。Agent 工具有明确参数 Schema，SQL 查询使用白名单、敏感字段黑名单、强制 LIMIT 和脱敏策略，避免大模型直接获得无限制数据库权限。第四，沉淀工程化 Agent 模式。本项目将 Agent 作为后端模块实现，具备控制器、服务、编排器、记忆、工具、消息队列和前端流式界面，形成较完整的 AI 应用开发闭环。

1.4 技术路线

系统采用前后端分离架构：前端为 Vue 3 + Vite + Element Plus + Pinia，负责登录注册、问卷列表、动态问卷填写、后台管理和 AI 助手聊天界面；后端为 Spring Boot 3.2 + Java 17，使用 MyBatis-Plus 操作 MySQL，Redis 支持缓存、限流、短期记忆与分布式锁，RabbitMQ 支持提交后异步处理和 Agent 异步任务。Agent 模块基于 LangChain4j，支持讯飞星火、DeepSeek、阿里云等 OpenAI-compatible 风格模

型配置，并通过 Hybrid 意图识别、ReAct 推理循环和业务工具实现从自然语言到系统能力的映射。

项目遵循“业务系统先稳定、智能能力再增强”的原则。普通问卷创建、提交和管理功能不依赖大模型即可运行；Agent 只是以工具形式调用已有业务能力，而不是绕过业务服务直接修改数据。对于高风险操作，例如创建问卷，SurveyBuildTool 设计为 preview 和 confirm 两阶段流程，先生成预览计划并保存到会话上下文，用户确认后才写入数据库。对于查询类操作，NaturalQueryTool 支持预设查询类型和 direct_sql，但 direct_sql 必须通过 SELECT 限制、禁用关键词、敏感字段检查、LIMIT 限制和脱敏处理。

二、Specs 规格文档

2.1 Product Spec

CC Survey 面向需要快速收集、分析和管理问卷数据的组织用户。产品角色主要包括普通用户、管理员和 AI 助手。普通用户可注册登录、查看可参与问卷、填写并提交答案；管理员可管理问卷、用户、答案、日志、统计数据并使用 AI 助手；AI 助手作为系统能力入口，根据自然语言自动路由到合适 Agent。

系统应支持用户认证、注册、密码重置、JWT Cookie 登录态管理；支持问卷模板创建、编辑、发布、预览、版本化和提交限制；支持动态字段类型 input、radio、checkbox、select、date、upload、multi-input；支持 showWhen 条件显示、computeRule 自动计算、validatorRule 自定义校验、extraInput 附加输入、options[].score 自动评分；支持文件上传和文件元数据管理；支持后台答案查看、操作日志查看、统计仪表盘；支持 AI 聊天、SSE 流式输出、会话列表、历史消息、会话标题修改和删除。

系统应提供统一响应结构 ApiResponse<T>，便于前端处理成功与错误；应具备基础安全能力，包括 BCrypt 密码、HttpOnly Cookie、JWT 校验、管理员接口鉴权、Redis 限流和 CORS 配置；应支持并发提交，避免达到提交上限时超发；应支持异步处理提交后任务，降低用户同步等待；应支持 Agent 记忆持久化，保证对话上下文可恢复。

2.2 Architecture Spec

后端包路径为 com.ccsurvey，核心模块包括 auth、user、survey、submission、file、log、statistics、agent、test。auth 处理认证和过滤器；survey 管理问卷模板与答案；submission 负责提交、评分、分布式锁和 RabbitMQ 事件；log 通过 @OperLog 与 AOP 自动记录操作；statistics 生成管理端统计数据；agent 封装

LangChain4j、多 Agent、意图识别、记忆、工具调用和流式响应；common 提供统一响应、异常处理、Redis、RabbitMQ、CORS、MyBatis-Plus、JWT 等公共配置。

前端采用 Vue 3 组合式架构，src/api 集中封装接口，src/stores/auth.js 保存认证状态，src/router/index.js 定义路由守卫。页面分为认证页、用户问卷流、后台管理流和 AI 助手流。公开路由包括登录、注册、忘记密码和公开问卷填写页；根路径 / 需要认证；/admin/* 需要管理员角色，包含问卷、用户、答案、日志、统计和 Agent 页面。Vite 开发环境将 /api/* 与 /files/* 代理至后端 8080 端口。

Agent 模块由 AgentController、AgentOrchestrator、IntentRouter、BaseAgent、AgentTool 等组成。IntentRouter 维护意图到 Agent 的映射：CREATE_SURVEY、UPDATE_SURVEY、SAVE_DRAFT 路由至 surveyAgent；ANALYZE_DATA 路由至 dataAnalysisAgent；QUERY_LOGS 路由至 logAnalysisAgent；NATURAL_QUERY 路由至 naturalQueryAgent；GENERAL_CHAT 和 UNKNOWN 路由至 routerAgent。BaseAgent 统一执行 ReAct 循环，最大迭代次数来自 agent.orchestration.max-iterations，默认配置为 5。

2.3 API Spec

认证接口位于 /api/auth，提供登录、注册、退出、当前用户、密码重置等能力；问卷公开接口用于获取问卷详情和提交入口；管理接口位于 /api/admin/*，由 AdminAuthFilter 保护，提供问卷、答案、用户、日志和统计管理。所有接口返回 ApiResponse<T>，业务异常由 GlobalExceptionHandler 转换为统一错误码。

AgentController 的主要接口包括：POST /api/agent/chat，非流式聊天，接收 conversationId、agentType、message 并返回 AgentResponse；POST /api/agent/stream，SSE 流式聊天，返回 text/event-stream；POST /api/agent/task，提交异步 Agent 任务到 RabbitMQ 并返回 taskId；GET /api/agent/conversations，获取当前用户会话列表；GET /api/agent/conversations/{conversationId}/messages，获取会话历史；DELETE /api/agent/conversations/{conversationId}，删除会话；PUT /api/agent/conversations/{conversationId}/title，更新会话标题且限制标题长度不超过 50 字符。

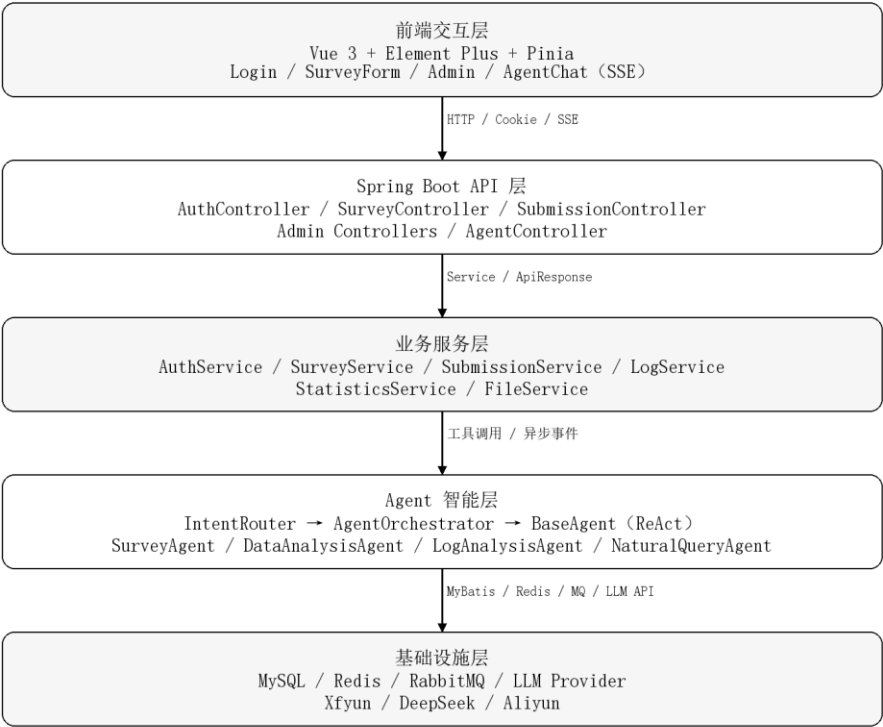
核心业务表包括 sys_user、survey_template、survey_answer、sys_oper_log、sys_file、sys_email_code、sys_config，以及 ai_conversation、ai_message 等 Agent 会话表。表使用自增 id 作为主键，同时使用 32 字符 uuid 作为外部引用。survey_template.schema_json 保存动态问卷结构，survey_answer.answer_data 保存用户答案，total_score 保存自动评分结果。

三、系统架构与设计

3.1 核心架构图

本系统采用“前端交互层—后端业务层—Agent 智能层—基础设施层”的四层结构进行设计。该架构既保留了传统问卷系统中用户管理、问卷管理、问卷填写、答案提交、统计分析等基础业务能力，又在此基础上引入 Agent 智能交互能力，使系统能够通过自然语言方式辅助用户完成问卷查询、数据分析、系统操作引导等任务。图中省略部分常规 CRUD 细节，重点突出课程要求关注的系统架构、Agent 交互流程与核心数据流。

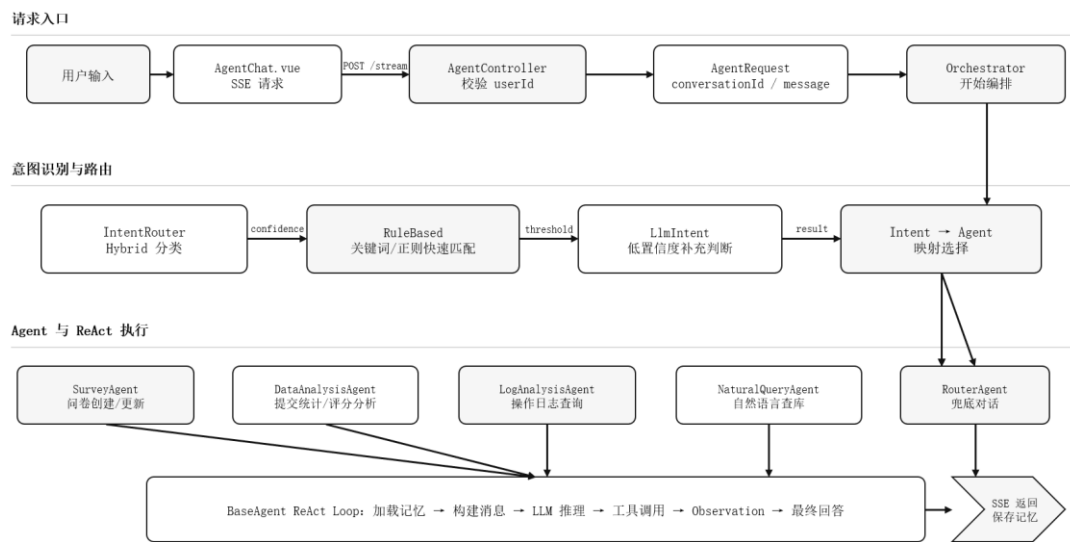
从整体上看，前端交互层负责向用户提供可视化操作界面；后端业务层负责处理系统核心业务逻辑、权限校验和数据访问；Agent 智能层负责对用户自然语言输入进行意图识别、任务路由和工具调用；基础设施层则为系统提供数据库、缓存、消息队列、文件存储以及大模型服务等支撑能力。各层之间职责清晰、耦合度较低，便于后续功能扩展和维护。



3.2 Agent 交互流程

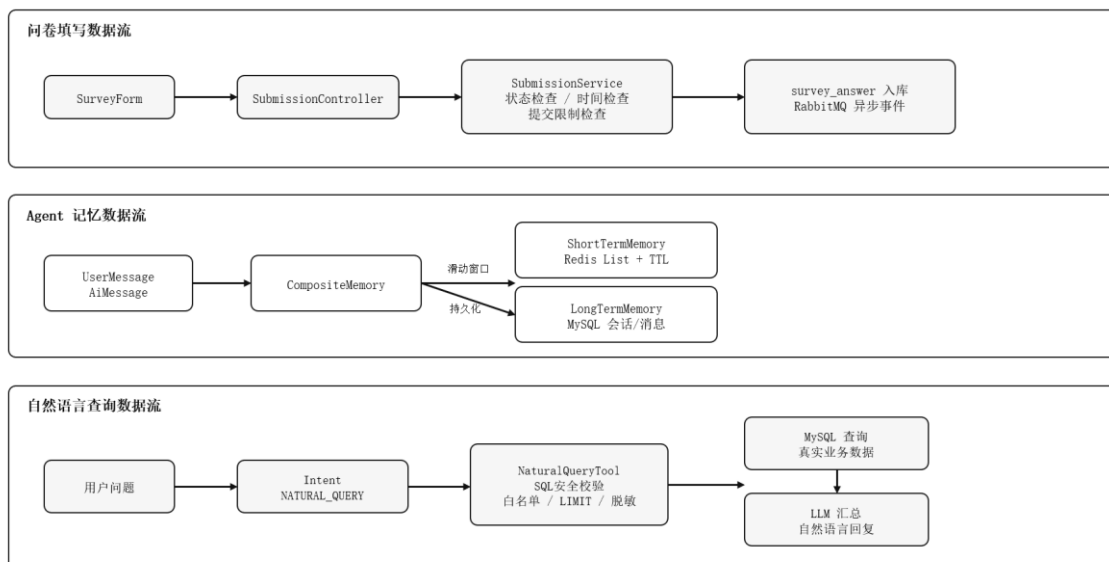
Agent 并不是脱离系统运行的独立脚本，而是后端 Agent 模块中的一条受控执行链。用户在 AgentChat 页面输入消息后，前端通过 fetch 调用 /api/agent/stream，并携带 Cookie 凭证，以保证请求能够继承当前登录用户的身份信息。请求首先经

过 JWT 认证过滤器解析用户身份，得到 `userId` 后进入 `AgentController`。控制器将用户输入、会话信息和用户身份等内容封装为 `AgentRequest`。编排器先调用 `IntentRouter` 完成意图识别，判断用户请求属于问卷查询、统计分析、系统帮助或普通问答等类型，再根据识别出的 `Intent` 选择对应 `Agent`。被选中的 `Agent` 会加载当前会话的历史记忆，结合预设系统提示词构造上下文，随后进入 `ReAct` 执行循环：当 LLM 判断需要访问系统数据或执行特定操作时，会发起工具调用，系统再执行对应 `AgentTool`，并将工具返回结果作为 `Observation` 写回消息列表。之后 LLM 基于新的上下文继续推理，直到生成最终回答并通过流式接口返回给前端。该流程使 `Agent` 的每一步执行都受到后端权限、路由和工具边界约束，避免其绕过业务系统直接操作数据。



3.3 数据流设计

问卷业务数据流与 `Agent` 数据流相互独立又可以互通。普通问卷提交主要由 `SubmissionService` 负责处理，包括问卷模板检查、开放时间检查、提交限制检查、自动评分以及答案入库等步骤。若问卷设置最大提交份数，系统会先读取 `Redis` 中的提交计数器；当计数器不存在时，再根据数据库中已有提交数量进行初始化。随后系统获取问卷级分布式锁，在锁保护下执行原子递增并判断是否超过上限；如果后续业务处理失败，则回滚 `Redis` 计数器，避免缓存数量与数据库实际提交数量不一致。提交成功后，系统通过 `RabbitMQ` 发布 `SubmissionEvent`，用于后续通知、统计更新或异步处理。`Agent` 查询数据时不会绕过业务数据源，而是通过受控工具访问 `survey_template`、`survey_answer`、`sys_user`、`sys_oper_log` 等表。这样既保证问卷提交链路的事务性和一致性，也使 `Agent` 能够基于真实业务数据进行查询、分析和回答。



四、关键实现与代码展示

4.1 Agent 核心循环

BaseAgent 是各专业 Agent 的共同执行框架，用于统一封装多轮对话、工具调用和记忆管理等通用逻辑。它在 `execute` 方法中依次完成加载历史记录、构建消息列表、执行 ReAct 循环和保存记忆，使不同类型的 Agent 不需要重复实现相同流程，只需关注自身提示词和可用工具。ReAct 循环内部按照 `agent.orchestration.max-iterations` 配置控制最大迭代次数，避免 LLM 在工具调用过程中陷入无限循环或长时间占用资源。每一轮都会调用 ChatModel 获取模型响应；若返回的 AiMessage 中包含 `toolExecutionRequests`，系统会先将 AI 消息加入上下文，再逐个执行对应工具，并把 `ToolExecutionResultMessage` 写回消息列表，作为下一轮推理的观察结果；若没有工具调用，则说明模型已经得到足够信息，当前回复被视为最终回答。该设计能够保证 Agent 执行流程的可控性和一致性。

```

public AgentResponse execute(AgentRequest request, AgentContext context) {
    List<ChatMessage> history = loadHistory(request.getConversationId());
    List<ChatMessage> messages = buildMessages(history, request, context);
    String response = executeReActLoop(messages, context);
    saveMemory(request.getConversationId(), request.getMessage(), response);
    return AgentResponse.success(request.getConversationId(), response);
}

protected String executeReActLoop(List<ChatMessage> messages, AgentContext
context) {

```

```

        int maxIterations = agentProperties.getOrchestration().getMaxIterations();
        while (iteration < maxIterations) {
            ChatResponse response =
chatModel.chat(ChatRequest.builder().messages(messages).build());
            AiMessage aiMessage = response.aiMessage();
            if (aiMessage.hasToolExecutionRequests()) {
                processToolCalls(aiMessage, messages, context);
                continue;
            }
            return fullResponse.toString();
        }
        return fullResponse + "\n\n(注： 处理过程较复杂， 已达到最大迭代次数)";
    }
}

```

4.2 工具定义

AgentTool 抽象类要求每个工具提供 `getName`、`getDescription`、`getParametersSchema` 和 `execute` 方法，并可转换为 `ToolDefinition`，从而统一工具在 Agent 体系中的注册和调用方式。工具参数采用 JSON Schema 风格描述，使 LLM 在发起工具调用时具有明确的参数约束，也便于后端进行校验、日志记录和审计。SurveyBuildTool 中的 `build_survey` 工具支持 `preview`、`confirm`、`update` 三种动作，其中创建问卷必须先执行 `preview` 生成预览内容，用户确认后才能执行 `confirm`，避免 Agent 直接修改业务数据。NaturalQueryTool 中的 `natural_query` 工具支持 `survey_submission_stats`、`top_surveys`、`user_submission_stats`、`survey_avg_score`、`login_stats`、`user_register_stats`、`surveys_no_submission`、`survey_detail`、`direct_sql` 等查询类型，可覆盖问卷提交、用户行为、登录注册和问卷详情等常见数据分析需求。通过工具抽象，系统将 LLM 的自然语言能力限制在受控接口内执行。

```

public Map<String, Object> getParametersSchema() {
    return createSchema(
        List.of(
            Property.of("query_type", "string", "查询类型"),
            Property.of("sql", "string", "direct_sql 类型的 SELECT 语句"),
            Property.of("survey_title", "string", "问卷标题"),
            Property.of("time_start", "string", "开始时间"),
            Property.of("time_end", "string", "结束时间"),

```

```
        Property.of("limit", "integer", "返回数量限制")
    ),
    List.of("query_type")
);
}
```

4.3 配置文件

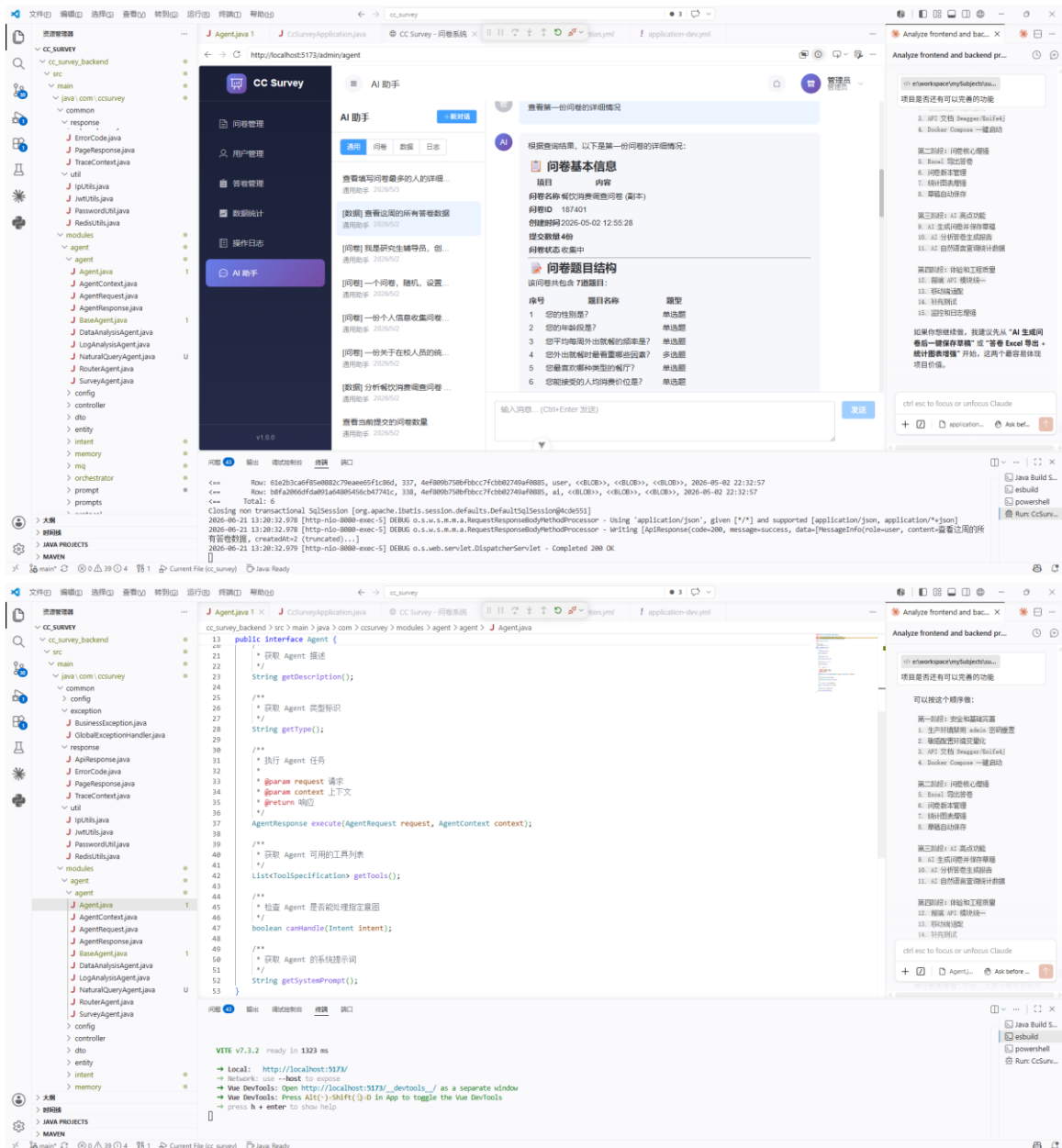
AgentProperties 通过 @ConfigurationProperties(prefix = "agent") 绑定 application.yml 中的配置项，集中管理 Agent 模块运行参数，主要包含 llm、memory、intent、orchestration 四组配置。其中，LLM provider 可在 xfyun、deepseek、aliyun 之间切换，便于根据环境或成本选择不同模型服务；短期记忆默认启用，窗口大小为 20，TTL 由配置控制，用于保留有限范围内的对话上下文；意图识别默认采用 hybrid 模式，以兼顾规则判断的稳定性和大模型理解能力；编排配置则包含最大迭代次数、超时时间以及是否允许并行执行等参数，用于约束 Agent 的执行边界。配置外部化后，开发、测试和生产环境可以使用不同模型、密钥、记忆策略和执行限制，而不需要修改业务代码，只需调整配置文件或环境变量即可完成切换，提高了系统部署灵活性和可维护性。

```
agent:
  llm:
    provider: ${LLM_PROVIDER:xfyun}
  memory:
    short-term:
      enabled: true
      window-size: 20
      ttl-minutes: 30
    long-term:
      enabled: true
      summarize-threshold: 50
  intent:
    classifier: hybrid
    confidence-threshold: 0.7
  orchestration:
    max-iterations: 5
    timeout-seconds: 60
    parallel-execution: true
```

4.4 AI IDE 使用截图

项目开发过程中，主要采用 VS Code 结合 Claude Code 的方式完成系统构建与功能完善。VS Code 作为统一开发界面，左侧可以查看前后端项目结构，中间区域

用于编辑 Spring Boot 后端代码、Vue 前端页面和配置文件，底部终端可同时运行后端 Maven 命令、前端 npm 命令以及测试命令。Claude Code 则用于辅助分析代码结构、生成或修改 Agent 模块逻辑、排查前后端接口联调问题，并根据实际项目文件给出修改建议。通过这种方式，开发、测试和调试可以集中在同一个界面内完成。例如，后端启动后可在终端观察接口日志和异常信息，前端启动后可直接测试页面交互与接口调用结果，同时结合 Claude Code 对控制台输出、报错堆栈和代码上下文进行分析。该开发方式提高了问题定位效率，也便于在实现 Agent 智能交互、问卷提交、统计查询等功能时持续进行前后端联调和验证。



五、测试与评估

5.1 功能测试

后端测试命令为 `./mvnw test`，可通过 `./mvnw test -Dtest=ClassName` 或 `./mvnw test -Dtest=ClassName#methodName` 运行指定测试类或测试方法，用于验证后端业务逻辑、配置加载和模块集成情况。前端可通过 `npm run build` 验证生产环境构建是否成功，通过 `npm run lint` 检查并自动修复 Vue/JS 文件中的格式或规范问题。在实际测试过程中，除命令行测试外，还结合浏览器页面操作和控制台日志观察进行联调验证。功能测试覆盖登录注册、管理员鉴权、问卷创建与发布、动态问卷渲染、答案提交、自动评分、提交限制、文件上传、后台答案查看、日志查询、统计面板和 Agent 聊天等流程。通过这些测试可以确认系统主要业务链路能够正常运行，前后端接口交互符合预期，Agent 模块也能够完成基本的对话、意图识别和工具调用。

5.2 Agent 行为评估

项目已有 `AgentModuleIntegrationTest`，用于验证 Agent 模块的基础运行能力和关键行为是否符合预期。测试内容包括 `AgentProperties` 配置加载、`ChatModel Bean` 注入、Agent 注册、`RouterAgent` 能力声明、规则基础意图识别、意图到 Agent 的映射、短期记忆存储等。典型用例包括：“帮我创建一个客户满意度调查问卷”应识别为 `CREATE_SURVEY`；“帮我分析一下问卷的提交数据统计”应识别为 `ANALYZE_DATA`；“查看最近一周的操作日志”应识别为 `QUERY_LOGS`；“你好，今天天气怎么样？”应识别为 `GENERAL_CHAT`。这些测试主要关注 Agent 在进入大模型推理前的路由与基础配置是否正确，能够保证不同用户输入被分发到合适的专业 Agent 或处理流程中。同时，短期记忆测试可以验证多轮对话上下文是否能够被保存和读取，为后续连续问答、数据追问和复杂任务执行提供基础。通过该部分评估，可以证明 Agent 模块不是简单调用 LLM，而是具备配置管理、意图分类、能力声明、路由分发和记忆维护等完整执行机制。

5.3 Benchmark 设计

由于项目运行依赖 MySQL、Redis、RabbitMQ 和外部 LLM，Benchmark 应分为本地业务基准和 Agent 端到端基准两类，以避免将数据库、消息队列和模型服务的耗时混在一起评估。本地业务基准主要测试问卷提交吞吐量、提交限制准确性、Redis 锁冲突下的失败率、RabbitMQ 异步消费延迟等指标，用于判断核心问卷业务在并发场景下是否稳定。Agent 端到端基准主要测试意图识别准确率、规则命中响应时间、LLM 分类响应时间、工具调用成功率、SSE 首字延迟和完整回答耗时，用于衡量智能交互链路的可用性和响应体验。简历文档中记录的目标表现为：规则层处理高频场景响应时间小于 100ms，Hybrid 意图识别平均响应时间小于 150ms，

准确率达到 95% 以上；SSE 首字响应目标小于 500ms。通过将业务链路和 Agent 链路分开评估，可以更准确地定位性能瓶颈，例如区分是数据库写入、Redis 锁竞争、RabbitMQ 消费延迟，还是外部 LLM 响应时间导致整体变慢，从而为后续优化提供依据。

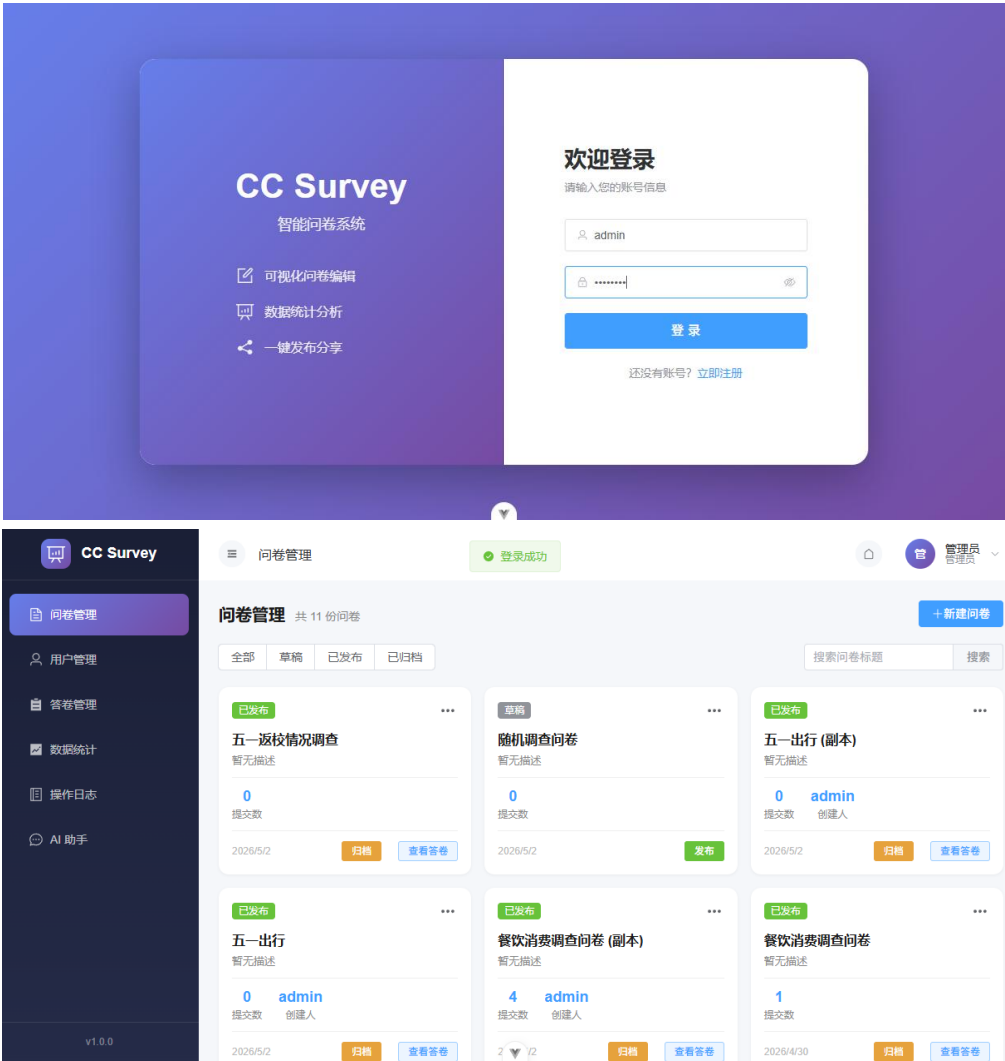
5.4 测试用例表

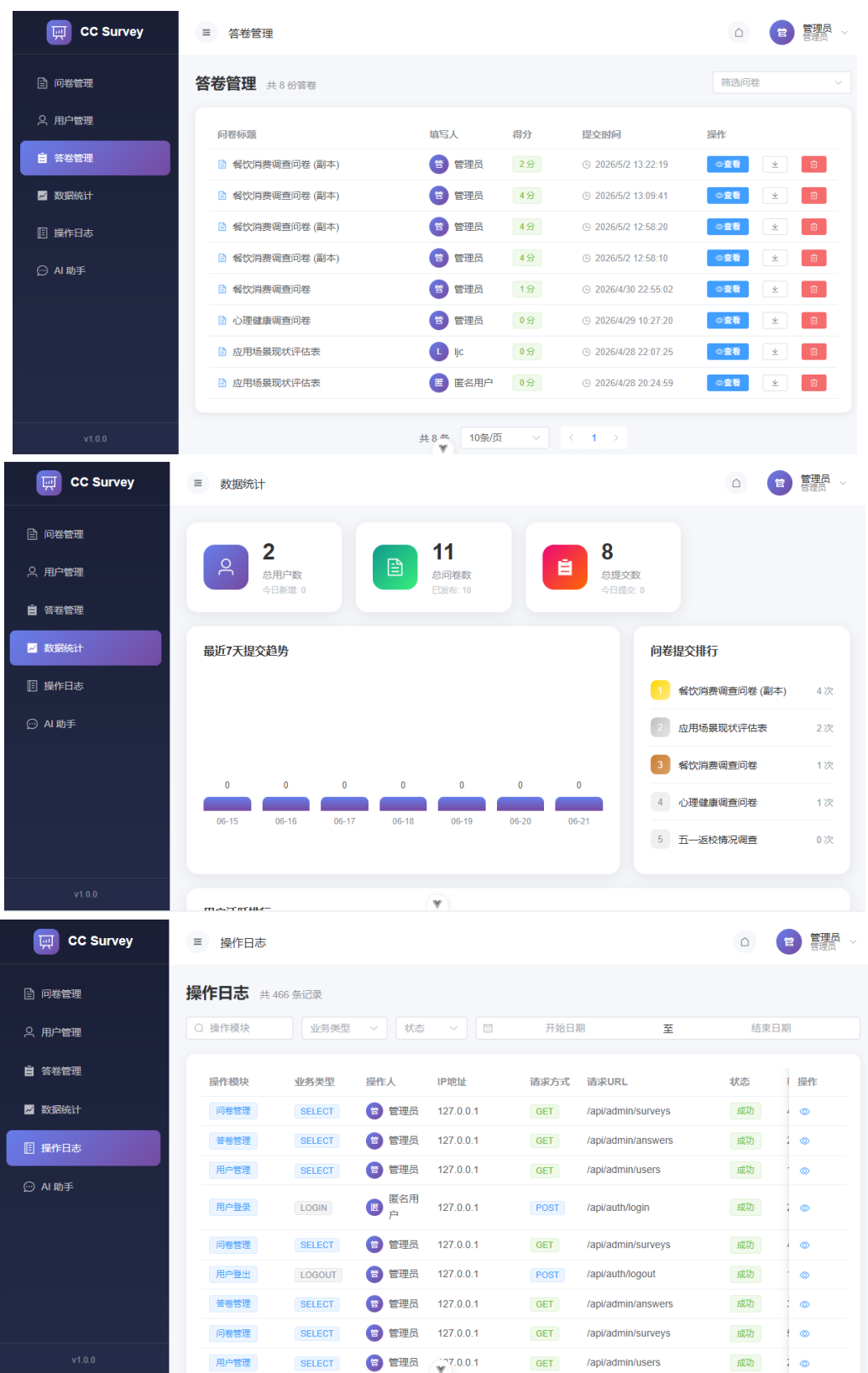
测试类别	输入/操作	期望结果
用户认证	输入正确账号和密码登录系统	登录成功，系统写入 JWT Cookie，并跳转到对应首页
权限控制	普通用户访问 /api/admin/* 管理接口	请求被管理员权限过滤器拒绝，无法访问后台资源
问卷管理	管理员创建并发布一份问卷	问卷保存成功，普通用户可在问卷列表中查看并填写
动态表单	打开包含单选、多选、日期、上传等字段的问卷	前端根据 schema_json 正确渲染表单控件
条件显示	选择触发 showWhen 条件的选项	联题目按规则显示或隐藏
问卷提交	用户填写完整问卷并提交	后端完成校验、评分和答案入库，前端进入提交成功页
提交限制	问卷提交数量超过最大提交份数	系统拒绝提交，并返回提交次数已达上限提示
文件上传	在问卷上传题中上传合法附件	文件上传成功，答案记录中保存对应文件信息
操作日志	管理员查看系统操作日志	系统展示用户操作记录，便于追踪关键管理行为
意图识别	输入“创建员工满意度调查”	识别为 CREATE_SURVEY，并路由到问卷创建相关 Agent
意图识别	输入“统计问卷提交情况”	识别为 ANALYZE_DATA，并路由到数据分析相关 Agent
自然查询	输入“上周提交量最高的问卷是哪一个”	路由到 NaturalQueryAgent，调用工具返回真实统计结果

5.5 Demo 截图说明

本部分通过系统实际运行截图展示 CC Survey 的主要功能界面和核心业务流程。首先，登录界面展示了系统的用户认证入口，用户输入账号和密码后进入对应角色页

面，后端通过 JWT Cookie 完成身份识别。问卷管理界面展示了管理员对问卷模板的维护能力，包括问卷列表查看、创建、编辑、发布和状态管理等操作，体现了系统对动态问卷配置的支持。答卷管理界面用于查看用户提交的问卷答案和相关信息，管理员可以基于提交记录进行结果追踪和数据核对。数据统计界面展示问卷数量、提交数量、用户数量等统计指标，辅助管理员了解系统整体运行情况。操作日志界面记录关键管理行为和用户操作信息，便于审计和问题追踪。AI 助手界面展示了系统的智能交互能力，用户可以通过自然语言进行问卷创建、数据查询、统计分析或系统使用咨询，系统通过 Agent 模块完成意图识别、工具调用和流式回答。通过这些截图，可以较完整地体现系统从登录认证、问卷管理、答卷查看、统计分析、日志审计到 AI 辅助交互的完整功能闭环。





CC Survey

操作日志

操作模块

业务类型

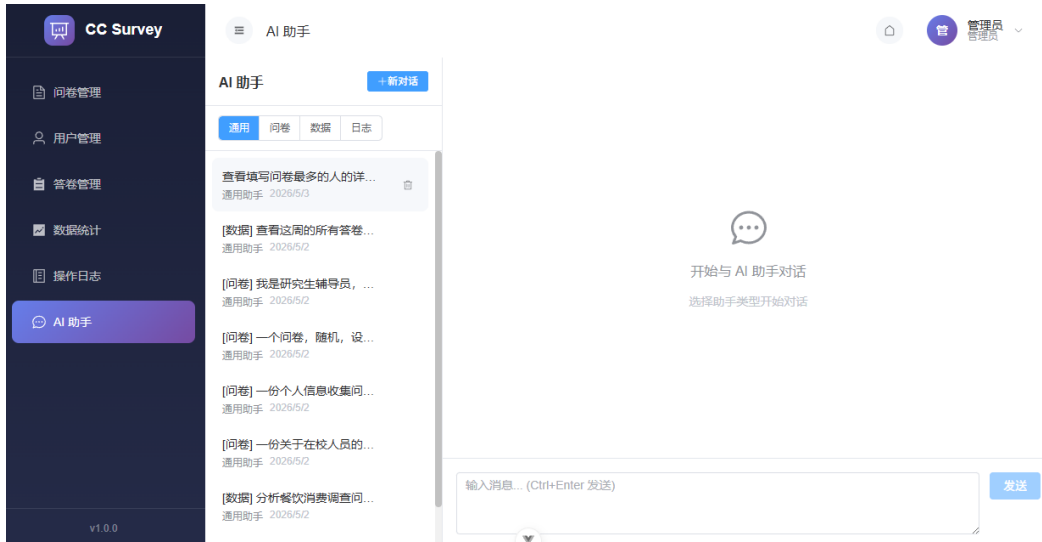
状态

开始日期

结束日期

共 466 条记录

操作模块	业务类型	操作人	IP地址	请求方式	请求URL	状态	操作
问卷管理	SELECT	管理员	127.0.0.1	GET	/api/admin/surveys	成功	查看
答卷管理	SELECT	管理员	127.0.0.1	GET	/api/admin/answers	成功	查看
用户管理	SELECT	管理员	127.0.0.1	GET	/api/admin/users	成功	查看
用户登录	LOGIN	匿名用户	127.0.0.1	POST	/api/auth/login	成功	查看
问卷管理	SELECT	管理员	127.0.0.1	GET	/api/admin/surveys	成功	查看
用户退出	LOGOUT	管理员	127.0.0.1	POST	/api/auth/logout	成功	查看
答卷管理	SELECT	管理员	127.0.0.1	GET	/api/admin/answers	成功	查看
问卷管理	SELECT	管理员	127.0.0.1	GET	/api/admin/surveys	成功	查看
用户管理	SELECT	管理员	127.0.0.1	GET	/api/admin/users	成功	查看



六、系统升级与扩展

6.1 可扩展架构

当前多智能体架构已经具备较好的扩展基础。新增业务能力时，可以按照“新增 Intent → 新增 AgentTool → 新增专业 Agent 或复用现有 Agent → 在 IntentRouter 中增加映射 → 前端增加交互入口”的方式迭代。该流程将意图识别、工具能力、Agent 执行和前端交互进行了分层拆分，避免新增功能时直接修改大量既有逻辑。由于 AgentTool 使用统一参数 Schema 描述工具名称、功能说明和参数结构，工具可以被多个 Agent 共享，也便于后续进行参数校验、日志审计和权限控制；由于 BaseAgent 已经抽象了历史记忆加载、消息构造、ReAct 循环和工具结果回写等通用流程，新增 Agent 时只需要关注自身系统提示词、可用工具列表和特定执行逻辑。该设计降低了功能扩展成本，使系统能够逐步扩展问卷生成、数据分析、日志查询、智能推荐等更多 Agent 能力。

6.2 下一阶段发展计划

。第一，完善可视化工作流。后续可将 Agent 执行过程中的 Thought、Action、Observation 以调试模式展示，普通用户默认不可见，管理员可用于追踪工具调用链路和定位异常。第二，引入 RAG 知识库。对于问卷设计方法、统计口径、学校课程规范、系统使用说明等相对固定的知识，可建立文档检索增强机制，减少大模型仅凭参数记忆回答带来的幻觉问题。第三，增强 Benchmark 平台。通过建立固定测试集，定期评估意图识别准确率、自然语言查询成功率、问卷生成质量、工具调用成功率和流式响应延迟。第四，完善权限粒度。当前系统管理员角色较粗，后续可拆分为问卷管理员、数据分析员、系统管理员等角色，使不同用户只能访问与职责相关的功能和数据。第五，增加更多导出与可视化能力，如问卷答案 Excel 导

出、图表下载、统计图联动筛选，以及由 Agent 自动生成问卷分析报告，从而提升系统在实际管理场景中的可用性。

6.3 AI 能力演进路径

短期阶段以工具调用稳定性为重点，保证 Agent 能准确选择工具、正确传参，并在工具执行失败时给出可理解的错误提示或恢复建议，避免直接中断对话。中期阶段可加入多 Agent 协同机制，例如由 SurveyAgent 负责生成问卷初稿，DataAnalysisAgent 评估题目是否便于后续统计分析，RouterAgent 负责汇总不同 Agent 的建议并返回给用户。长期阶段可引入“计划—执行—反思”机制，让 Agent 在生成问卷后自动检查题目是否覆盖目标维度、是否存在诱导性表述、是否缺少评分规则或逻辑跳转，并给出优化建议。对于自然语言查询能力，可从当前预设 query_type 的方式逐步升级为受控 SQL 生成器，同时结合数据库元数据、查询代价估计、白名单约束和审计日志，保证查询灵活性的同时降低误查、慢查询和越权访问风险。通过分阶段演进，系统可以从可用的智能助手逐步发展为更可靠的业务协同 Agent。

七、课程总结

7.1 个人学习收获

通过本项目，我对 AI Agent 应用开发的理解从“调用大模型生成文本”转向“让大模型在受控边界内调用系统能力”。传统 Web 项目更关注接口设计、数据库操作和页面交互，而 Agent 项目还需要额外关注意图识别、上下文记忆、工具 Schema、推理循环、模型失败兜底和安全审计等问题。LangChain4j 提供了较好的 Java 生态集成能力，使 Spring Boot 项目能够以工程化方式接入大模型，而不是把 AI 逻辑散落在独立脚本中。在实现过程中，我也认识到 Agent 并不是简单增加一个聊天框，而是需要与认证、权限、业务服务、数据库和日志系统结合，才能真正服务于实际业务。通过本项目实践，我进一步理解了 AI 能力落地时“可控、可测、可维护”的重要性，也提升了前后端联调、后端模块设计和智能工具调用设计能力。

7.2 工程思维转变

最大的变化是从“功能实现思维”转向“系统边界思维”。在传统功能开发中，更多关注某个接口或页面是否能够完成需求；而在 Agent 系统中，模型具备一定自主决策能力，因此更需要提前明确执行边界。Agent 能力越强，越要规定哪些操作可以由模型直接执行，哪些必须经过用户确认；哪些数据可以查询，哪些字段需要脱敏；工具调用失败时是否继续，最大迭代次数如何限制；历史记录保留多少，如何在上下文连续性和用户隐私之间取得平衡。项目中的 preview-confirm 机制、SQL 白名单校验、Redis/MySQL 双层记忆、统一 ApiResponse 响应结构和过滤器

链权限控制，都是这种边界思维的体现。通过这些设计，系统既能利用大模型提升交互效率，又能避免模型越权操作、错误调用工具或产生不可控的数据访问风险。该过程让我认识到，AI Agent 工程化的关键不只是让模型“能做事”，更重要的是让它在安全、清晰、可追踪的范围内做事。

7.3 对课程的建议

建议课程继续强化“AI + 真实业务系统”的实践，而不仅停留在单轮聊天 Demo。当前大模型应用已经逐渐从简单问答发展到工具调用、业务编排和多 Agent 协作，可以引导同学将 AI 能力接入具体业务流程，例如数据查询、表单生成、流程审批或知识库问答等。评估方面，可以增加若干标准维度，例如意图识别准确率、工具调用成功率、端到端响应时间、安全拦截效果、日志可追踪性和报告可复现性等，使不同项目之间更容易比较。展示方面，也可以提供统一的截图或录屏提交规范，让同学在演示时更聚焦关键链路，而不是只展示最终页面效果。建议增加 MCP、RAG 和工作流编排的对比实践，帮助大家理解不同技术方案在开发成本、生态支持、可控性和业务边界上的差异。